

Practice Set: 09

Section 1: Conceptual & Practical Questions (1–10)

Question 1: Define a **String** in C++. Contrast legacy C-style character arrays (`char[]`) with modern `std::string` across memory location, resizing capability, null-termination, and copying semantics.

Question 2: Explain the memory representation of **C-Style Strings** (`char[]`). What is the purpose of the null terminator (`'\0'`), and what problems occur if it is missing?

Question 3: What is the **Buffer Overflow Hazard** associated with legacy C-string utility functions like `strcpy()` and `strcat()`?

Question 4: Explain what three internal fields a `std::string` object encapsulates in memory.

Question 5: Differentiate between accessing string elements using the subscript operator `str[i]` and the member function `str.at(i)`.

Question 6: Detail the behavior and return values of the following `std::string` member functions: `length()/size()`, `append()`, `find()`, and `substr()`.

Question 7: Explain the difference in delimiter handling between `cin >> str;` and `getline(cin, str);`.

Question 8: Explain the **Stranded Buffer Trap** that occurs when mixing `cin >>` followed by `getline()`. How does `cin.ignore()` fix this issue?

Question 9: What special constant does `str.find()` return when a searched pattern is not found in a `std::string`?

Question 10: Write a complete C++ program that prompts the user for their age (`cin >>`) and full name (`getline()`), flushes the input buffer properly with `cin.ignore()`, and demonstrates substring extraction and pattern searching with `std::string`.

Section 2: Interview & Viva Questions (11–20)

Question 11 (Viva): How many total bytes in RAM are allocated for the C-style string literal `char str[] = "Hello";`?

Question 12 (Viva): What exception is thrown by `std::string::at()` when an index is out of bounds?

Question 13 (Viva): What header file must be included to use modern `std::string` in C++?

Question 14 (Viva): Which legacy C header provides functions like `strlen()` and `strcat()`?

Question 15 (Viva): Why is `getline(cin, str)` preferred over `cin >> str` when reading sentences or full names?

Question 16 (Viva): What is the numeric ASCII value of the null-terminator byte (`'\0'`)?

Question 17 (Viva): What happens if you call `getline()` immediately after `cin >> number` without using `cin.ignore()`?

Question 18 (Viva): Where does `std::string` allocate memory for larger character buffers?

Question 19 (Viva): What is the return type of `str.find()` in modern C++?

Question 20 (Viva): What does the function call `str.substr(0, 4)` return?

Answers & Explanations

Answer: 01

Definition: A String in C++ is a sequential collection of characters used to store and manipulate textual data.

Comparison Matrix:

Memory Location: C-style strings reside in stack arrays or read-only data segments; `std::string` manages a dynamic heap buffer (with Short String Optimization on the stack).

Resizing Ability: C-strings have a fixed, static size; `std::string` resizes dynamically at runtime.

Null Termination: C-strings require manual null-termination; `std::string` manages termination automatically.

Copying Semantics: C-strings require manual, error-prone `strcpy`; `std::string` supports deep copying and efficient `$O(1)$` move semantics.

Answer: 02

Memory Layout: C-strings are stored as 1-D contiguous character arrays ending with an invisible ASCII 0 byte called the null terminator ('\0').

Role & Omission Impact: Functions like `strlen()` and `cout` rely on '\0' to know where the string ends. If missing, reading continues into adjacent memory addresses, producing garbage characters or triggering segmentation faults.

Answer: 03

Legacy functions like `strcpy()` and `strcat()` perform no boundary checks. Writing data larger than the allocated target array overwrites adjacent stack RAM, leading to memory corruption and severe security vulnerabilities.

Answer: 04

Pointer (char*): Memory address pointing to the character buffer located on the heap.

Size (size_t): The exact number of valid characters currently stored.

Capacity (size_t): The total allocated memory buffer available before reallocation is required.

Answer: 05

str[i]: Directly accesses the character at index `i` without bounds checking for maximum speed.

str.at(i): Performs safe bounds checking and throws a `std::out_of_range` exception if `i` is outside the valid range.

Answer: 06

length() / size(): Returns the total count of characters currently stored in the string.

append() / +=: Appends characters to the existing string, automatically reallocating heap memory if needed.

find("pattern"): Searches for the first occurrence of a substring and returns its starting index, or `std::string::npos` if not found.

substr(pos, count): Returns a newly extracted sub-string starting at index `pos` spanning `count` characters.

Answer: 07

cin >> str;: Reads single-word tokens, stopping immediately at the first whitespace character (space, tab, or newline).

getline(cin, str); Reads an entire line of text, including spaces, until an enter key ('\n') is encountered.

Answer: 08

The Problem: When reading a number using `cin >> age`, the user types the number followed by Enter, leaving the newline character ('\n') stranded in the stream buffer. A subsequent `getline()` call reads that leftover newline and terminates immediately, storing an empty string and skipping user input.

The Solution: Calling `cin.ignore(numeric_limits<streamsize>::max(), '\n');` discards the leftover newline byte, allowing `getline()` to wait for actual user input.

Answer: 09

It returns **std::string::npos** (a constant representing an invalid index position).

Answer: 10

```
#include <iostream>
#include <string>
#include <limits>
using namespace std;
int main()
{
    int age;
    string fullName;

    cout << "Enter Age: ";
    cin >> age;

    // Discard lingering newline from input buffer
    cin.ignore(numeric_limits<streamsize>::max(), '\n');

    cout << "Enter Full Name: ";
    getline(cin, fullName);

    cout << "\nUser: " << fullName << " (Age: " << age << ")" << endl;

    // Substring and Search Demonstration
    string sub = fullName.substr(0, 4);
    cout << "First 4 chars: " << sub << endl;

    size_t pos = fullName.find(" ");
    if (pos != string::npos) {
        cout << "First space detected at index: " << pos << endl;
    }
}
```

```
    return 0;  
}
```

Answer: 11

6 bytes (5 bytes for the letters 'H', 'e', 'l', 'l', 'o' plus 1 byte for the null terminator '\0').

Answer: 12

std::out_of_range.

Answer: 13

The **<string>** header file.

Answer: 14

The **<cstring>** header file.

Answer: 15

getline() reads strings containing spaces and tabs up to a newline, whereas cin >> terminates at the first whitespace character.

Answer: 16

The ASCII value is **0**.

Answer: 17

getline() reads the stranded '\n' character remaining in the buffer and immediately returns an empty string without waiting for the user to type.

Answer: 18

On the **Dynamic Heap**.

Answer: 19

size_t (an unsigned integral type).

Answer: 20

A substring consisting of the first **4 characters** starting from index 0.